CLOUD & VIRTUALISATION | TEK-UP UNIVERSITY

Lab 01

Installing and managing virtual machines with VirtualBox

Student

Assil Mhadhbi

Class

Ing1, Groupe C (evening course)

Date

15 October 2026

About this report

In this lab I installed a Linux virtual machine in VirtualBox, connected it to my Windows machine with Guest Additions and a shared folder, and then practised the three ways of protecting or copying a VM: snapshots, clones and export/import.

Two things are different from the lab handout. First, I used **Red Hat Enterprise Linux 10** instead of Ubuntu, so the `apt` commands became `dnf` commands and I had to deal with a Red Hat subscription. Second, I did the snapshot before the clones and the export, because a linked clone needs a snapshot to exist. The report follows the order I really worked in.

One part is not finished: the shared clipboard between Windows and the VM still does not work. Section 9 explains everything I checked.

1. My setup

Item	What I used
Host machine	Windows, with Oracle VirtualBox 7.2.20 (type 2 hypervisor)
Guest system	Red Hat Enterprise Linux 10 (rhel-10.0-x86_64-dvd.iso)
VM name	VM1-RedHat-Desktop
Processor and memory	2 vCPUs, 4096 MB of RAM
Disk	20 GB virtual disk (VDI)
Network	One adapter in NAT mode (Intel PRO/1000 MT Desktop)
Graphics controller	VMSVGA

Since I installed Red Hat, Method 2 for Guest Additions in the handout (installing from the repositories with `apt`) does not exist for me. I used Method 1, the ISO, only.

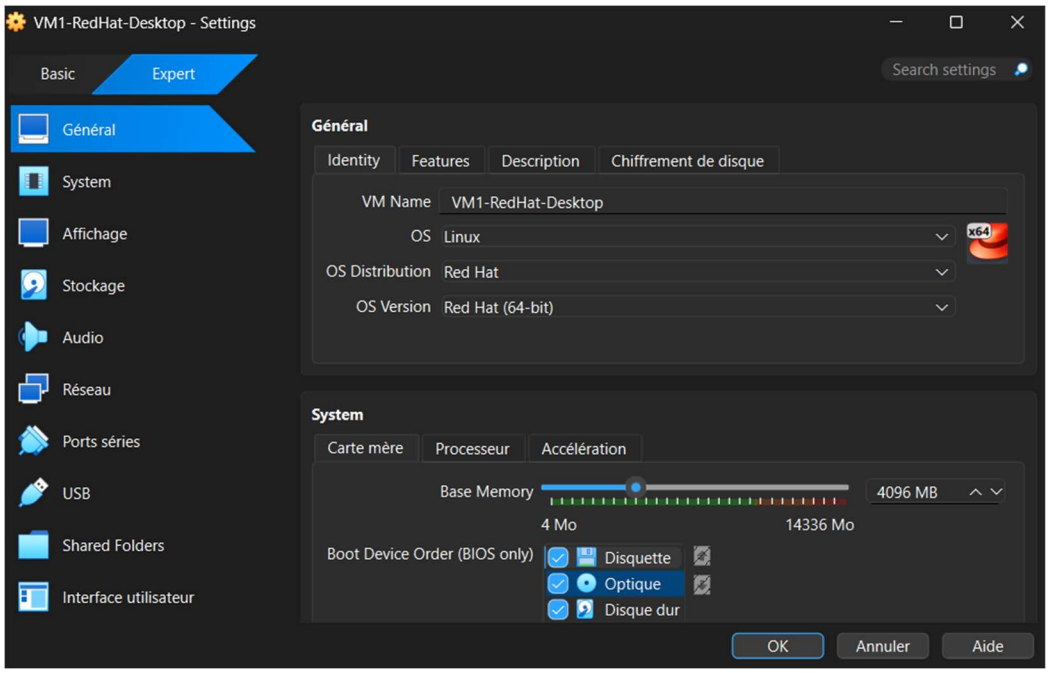


Figure 1. Settings of VM1-RedHat-Desktop in VirtualBox (General and System tabs).

2. Creating the VM and getting it to boot

I created the VM with the New wizard and skipped the unattended installation, as the handout asks, so that I could install the system by hand. I gave it 2 vCPUs, 4096 MB of RAM and a 20 GB disk, and attached the RHEL 10 DVD ISO.

> **Problem I ran into**
>
> When the installation ended, the VM switched itself off, and every time I started it again the screen stayed completely black. I fixed it by going through the display settings (VMSVGA as graphics controller, 3D acceleration off) and by checking the boot order and the ISO in the storage settings. After that the VM booted normally.

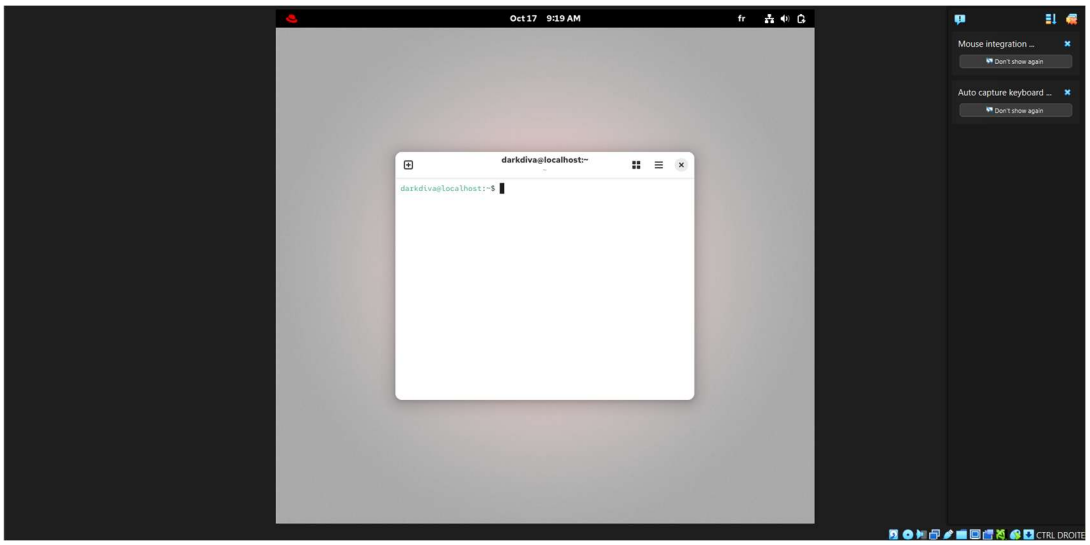


Figure 2. The Red Hat desktop running inside VirtualBox after the first successful boot.

3. Getting dnf to work

Before I could install anything, `sudo dnf update` stopped with the error "There are no enabled repositories". The network was fine (a ping to 8.8.8.8 worked) and the VM was registered with Red Hat, but `subscription-manager repos --list` returned nothing.

The real cause was on my Red Hat account: it did not have an active subscription yet. The system uses Simple Content Access, so there is nothing to attach by hand. Once the free **Red Hat Developer Subscription for Individuals** appeared as active on my account (valid from 2026-10-10 to 2027-10-10), I registered the VM again. I also lost a little time because I typed `refrech` instead of `refresh` at one point.

```
sudo subscription-manager unregister
sudo subscription-manager clean
sudo subscription-manager register --username <my username>
sudo subscription-manager refresh
sudo subscription-manager repos --list
sudo dnf clean all
sudo dnf update -y
```

After this, the repository list was filled in and `dnf` worked.

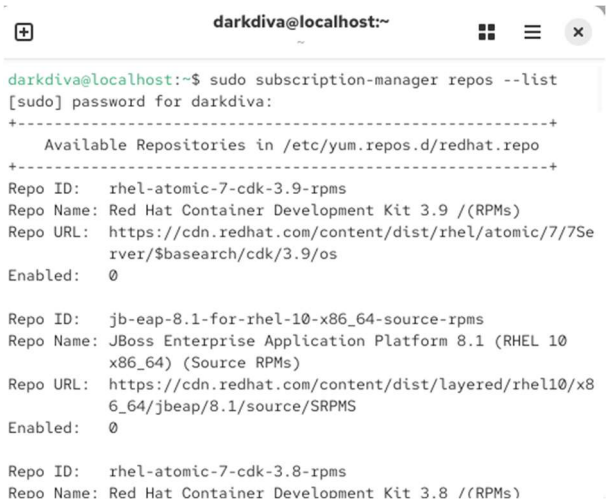


Figure 3. subscription-manager repos --list now returns the available repositories (part 1).

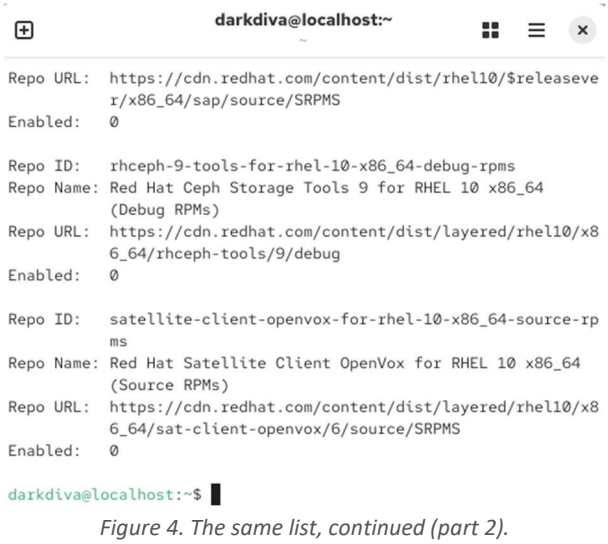


Figure 4. The same list, continued (part 2).

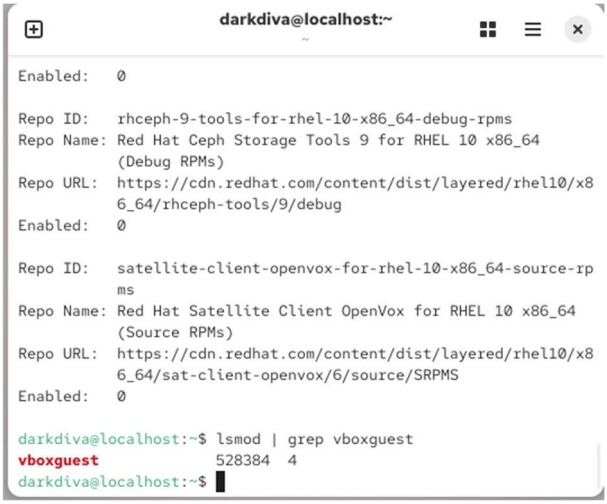


Figure 5. The same list, continued (part 3).

4. Installing Guest Additions

Guest Additions are drivers that let the VM and Windows work together: shared folders, shared clipboard and better display. They need compilers and the kernel headers, which on Red Hat come from `dnf`. I updated the system, rebooted, then installed the tools (this replaces `build-essential dkms linux-headers` from the handout):

```
sudo dnf update -y
sudo reboot
sudo dnf install -y gcc make perl bzip2 tar elfutils-libelf-devel \
    kernel-devel-$(uname -r) kernel-headers
```

The option **Insert Guest Additions CD image** did not give me a mounted disc: `cd` said the folder did not exist and `mount` said it could not open the block device. So I attached `VBoxGuestAdditions.iso` myself from the VirtualBox installation folder on Windows (Devices > Optical Drives > Choose a disk file), then mounted it and ran the installer:

```
sudo mkdir -p /media/cdrom
sudo mount /dev/sr0 /media/cdrom
cd /media/cdrom
sudo sh VBoxLinuxAdditions.run
sudo reboot
```

After the reboot, `lsmod | grep vboxguest` shows that the module is loaded, so the installation worked.



Figure 6. The vboxguest kernel module is loaded.

5. Sharing a folder between Windows and the VM

I made a folder on Windows and put a test file in it. Then in the VM settings (Shared Folders) I added it with the folder name `shared` and ticked **Make Permanent**. Inside the VM I mounted it by hand:

```
sudo mount -t vboxsf shared /mnt
cd /mnt
ls
```

The file from Windows appears in the VM. It is called `test.txt.txt` because Windows was hiding the real extension when I named it, which is harmless.



Figure 7. Manual mount of the shared folder: the Windows file is visible in /mnt.

What happens after a reboot (question 23). A mount made with the `mount` command only lasts until the VM is restarted. To keep the share, it has to be listed in `/etc/fstab`, the file Linux reads at boot to know what to mount. I added this line at the end:

```
shared /mnt vboxsf defaults,x-system.automount 0 0
```

Then I ran `sudo systemctl daemon-reload` and rebooted. The folder is now there without any manual command.



Figure 8. ls /mnt with the shared folder available.

6. Snapshots

A snapshot saves the state of a VM at one moment so I can come back to it later. I did this before the clones because a linked clone depends on a snapshot.

I took a snapshot named **Avant-test** on 13 October 2026 from the Snapshots tab. To test it, I started the VM and created a file with `touch ~/aftersnapshot.txt`. The file shows up in the home folder.

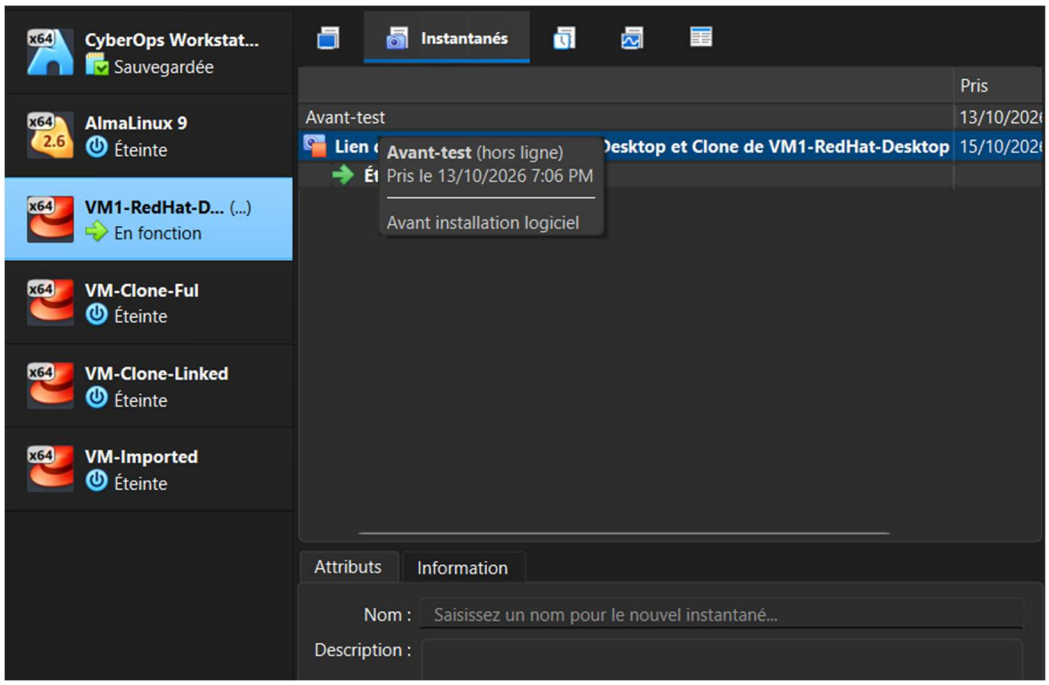


Figure 9. The Snapshots tab with Avant-test. The VM list on the left also shows the clones and the imported VM.

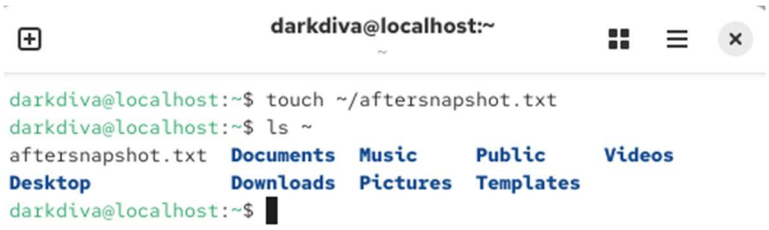


Figure 10. The test file aftersnapshot.txt exists before the restore.

Then I shut the VM down and restored the snapshot (right-click on the snapshot, then Restaurer).

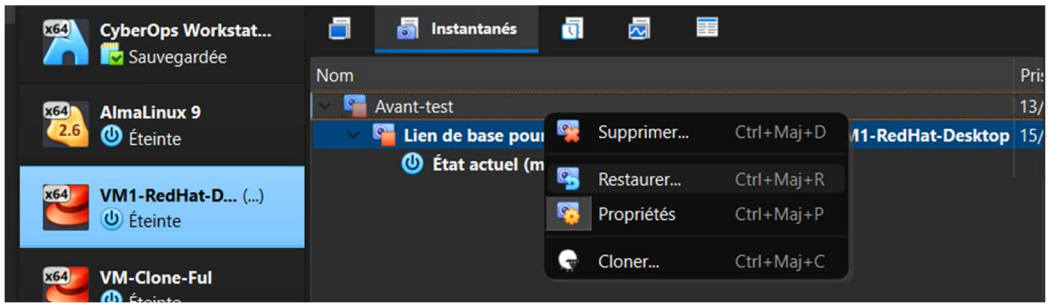


Figure 11. Restoring the Avant-test snapshot.

After the restore I started the VM again and listed my home folder. The file is gone, which proves that the VM went back to its earlier state.

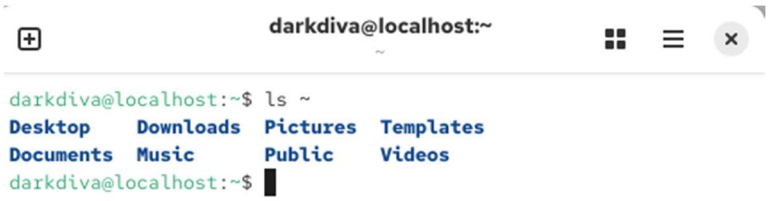


Figure 12. ls ~ after the restore: aftersnapshot.txt no longer exists.

7. Cloning

I made two clones of VM1-RedHat-Desktop with a right-click and **Cloner**: a full clone and a linked clone (called VM-Clone-Linked). Both appear in the VM list shown in Figure 9.

	Full clone	Linked clone
What it is	A completely independent copy, the disk is duplicated	Shares the disk of the original through a snapshot
Good point	Works on its own, even if the original is deleted	Created quickly and takes very little space
Bad point	Takes more disk space and more time	Depends on the original VM and its snapshot

8. Exporting and importing a VM

Exporting turns a VM into a file I can move to another computer. VirtualBox offers OVF (several files) and OVA (everything in one archive). I chose **OVA** because a single file is easier to handle.

With the VM switched off, I used **Fichier > Exporter un appareil virtuel**, picked the VM, chose OVA and kept the MAC addresses of all network adapters. The export finished and gave me one `.ova` file. Then I used **Fichier > Importer un appareil virtuel**, selected that file, renamed the new VM to `VM-Imported` and asked VirtualBox to generate new MAC addresses so the copy does not clash with the original.

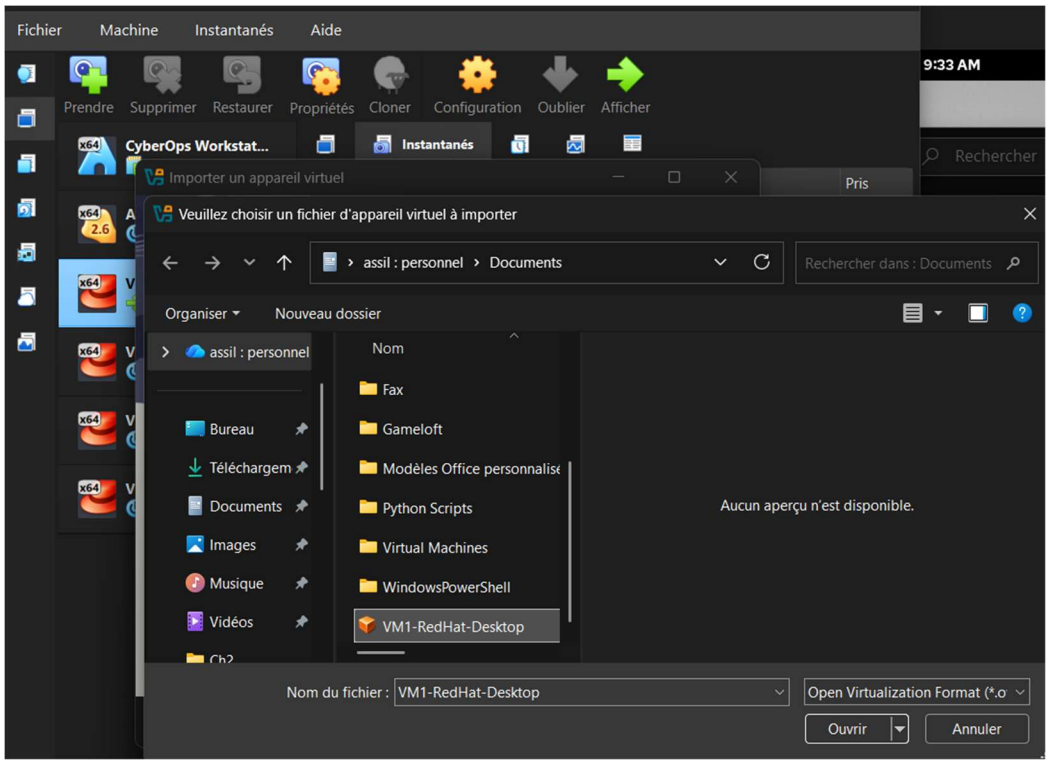


Figure 13. Choosing the .ova file in the import wizard.

> **Problem I ran into**
>
> The imported VM started and I could log in, but `ls /mnt` answered **No such device**. The `/etc/fstab` line was still there because it lives inside the virtual disk, but the shared folder itself is part of the VM's VirtualBox settings, and those settings are not carried inside the exported file. So the VM was looking for a folder that did not exist.



Figure 14. ls /mnt on the imported VM before the fix.

How I fixed it:

1. I shut the VM down with `sudo poweroff`.

2. In VirtualBox I opened the settings of VM-Imported, went to the shared folders section and added my Windows folder again, with the name `shared` (exactly the name used in fstab) and the permanent option ticked.

3. I started the VM again and ran `cd /mnt` then `ls`. The shared file was back, and `lsmod` still showed the `vboxguest` and `vboxsf` modules, so the import is a success.

9. The shared clipboard (not working)

I set the shared clipboard to **Bidirectional** in the VirtualBox menu (Périphériques > Presse-papiers partagé) and in the VM settings. Copy and paste works inside the VM, but not between Windows and the VM. Since this is the only part that failed, I spent time finding out why:

What I checked	What I found
VirtualBox version on Windows	7.2.20 r175154
Guest Additions version in the VM	7.2.20 r175154, the same version, so it is not a version mismatch
Guest Additions module	Loaded (`lsmod | grep vboxguest`)
Clipboard service (`pgrep -a VBoxClient`)	`VBoxClient --clipboard` is running
Setting in VirtualBox	Bidirectional
Type of session (`echo $XDG_SESSION_TYPE`)	wayland (VBoxClient itself reports XWayland)

At one point I had two clipboard services running at the same time because I started one by hand while the system had already started one, so I rebooted to get back to a single one. It did not change the result.

The version, the service and the setting are all correct, so I do not think the installation is the problem. My best explanation is the **Wayland** session: RHEL 10 only offers Wayland, and the VirtualBox shared clipboard is known to be unreliable there. I could not prove it, because there is no Xorg session on RHEL 10 to compare with. The thing I would try next is the same test on my AlmaLinux 9 VM using the "GNOME on Xorg" login option. In the meantime, the shared folder from section 5 is how I exchange files and text with the VM.

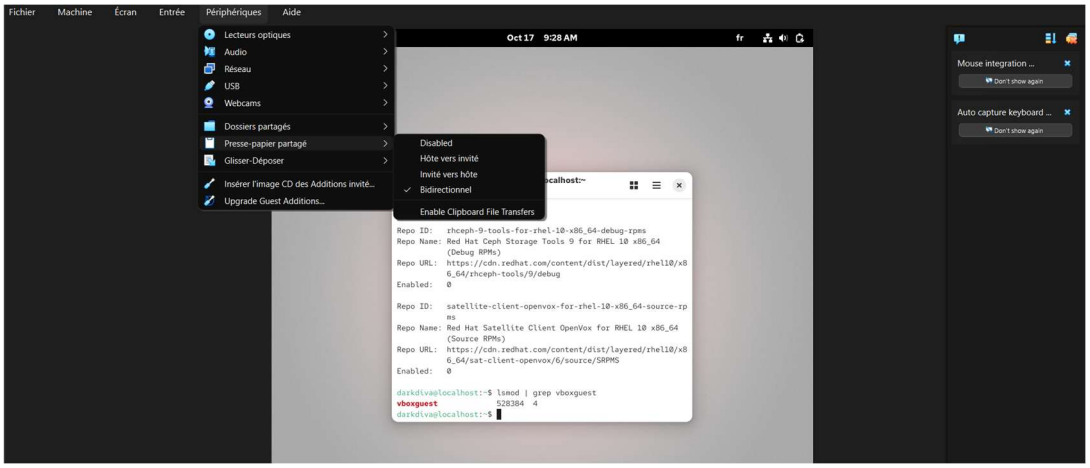


Figure 15. The VM menu showing the shared clipboard option.



Figure 16. VirtualBox version on the Windows host: 7.2.20.

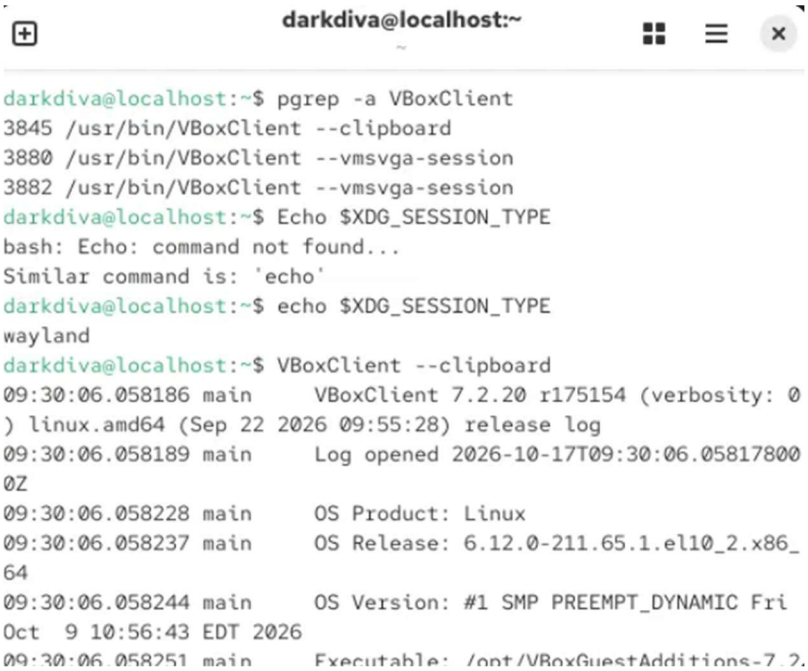


Figure 17. The clipboard service is running and the session type is wayland.

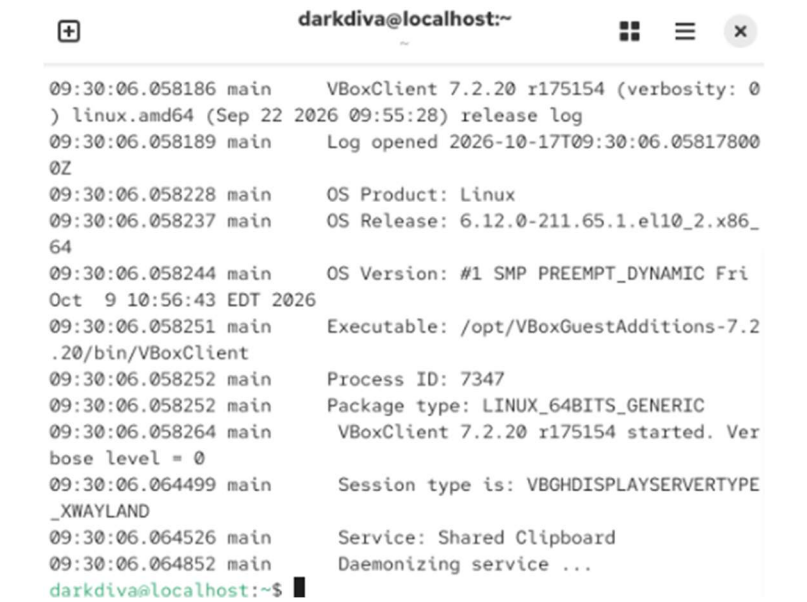


Figure 18. VBoxClient log: Guest Additions 7.2.20, Shared Clipboard service started.

10. Summary

Task	Result	Comment
VM creation and installation	Done	Red Hat 10 instead of Ubuntu; black screen fixed
Guest Additions	Done	ISO method only; vboxguest module loaded
Shared folder (manual and permanent)	Done	fstab option x-systemd.automount
Snapshot and restore	Done	Avant-test; the test file disappeared after the restore
Cloning (full and linked)	Done	Both clones created
Export and import (OVA)	Done	Shared folder had to be added again after the import
Shared clipboard	Not working	Setting, service and versions are correct; probably a Wayland limitation

11. What I learned

The biggest lesson was that the handout is written for one system and I was on another. Switching from Ubuntu to Red Hat meant `dnf` instead of `apt`, and it also meant that without an active subscription I could not install a single package. I learned to read the error message first (no enabled repositories) and to check the network, the registration and the account one by one.

I also understood why `fstab` matters: a manual mount disappears at reboot, and one wrong letter in the options line breaks the automount. Export and import taught me that an exported VM does not carry everything with it, since the shared folder had to be set up again. And between a full and a linked clone, the choice is simply space and speed against independence.

The clipboard is the one thing I could not solve. I would rather say so and show my checks than pretend it works.